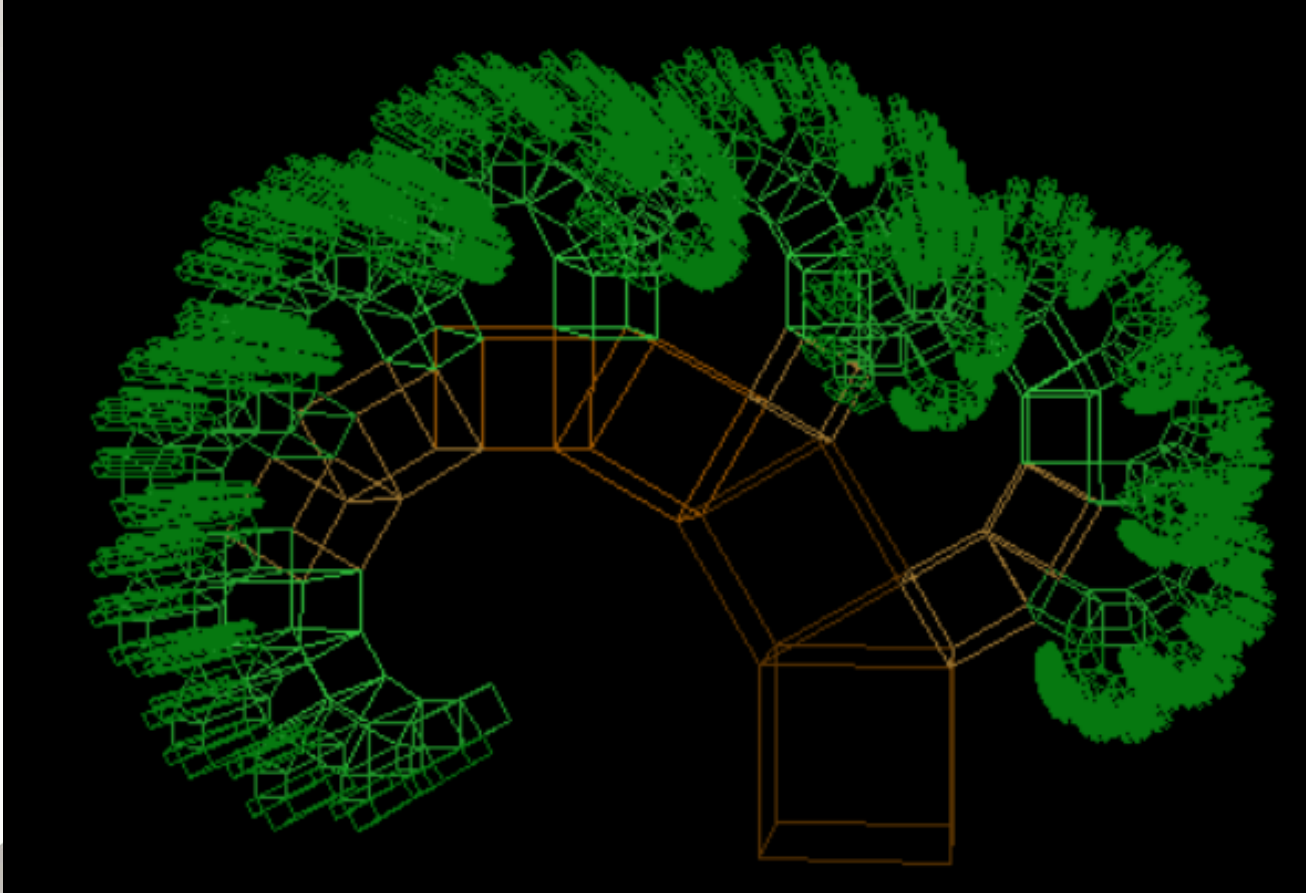




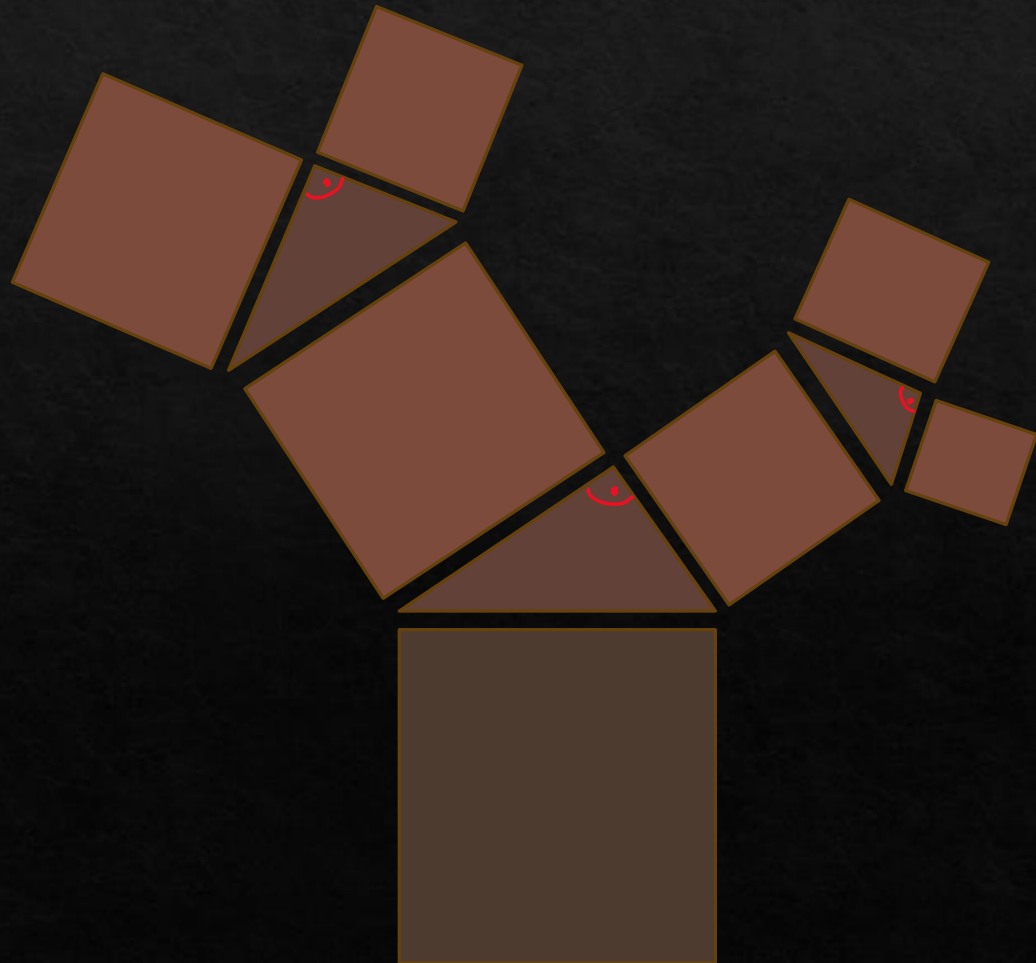
Pythagorasbaum 3D

Projektvorstellung PaCo

Patrick Denz, Sofie Birkle, Tomke Velten
22.01.2026

Pythagorasbaum

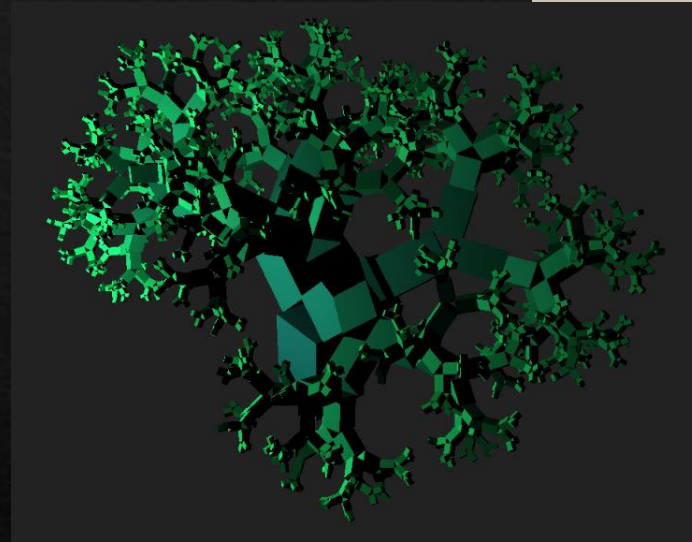
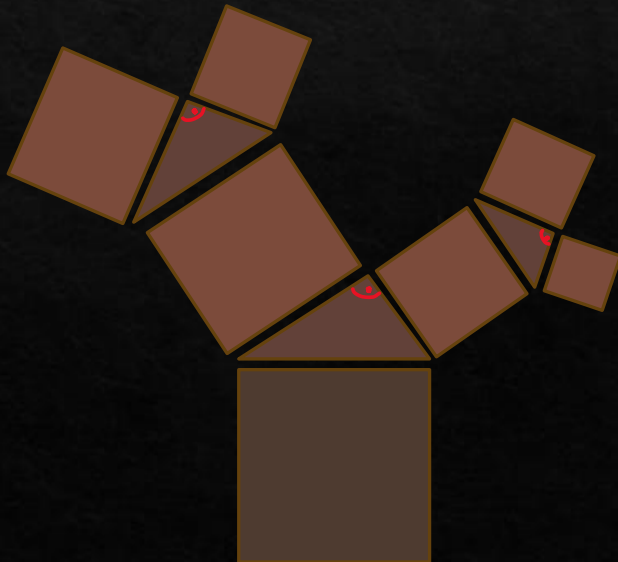
- ◇ Dreiecksgeometrie
 - Satz des Pythagoras
- ◇ ursprünglich 2D



Pythagorasbaum

=> 3D:

- ◇ Dreiecksgeometrie
 - Satz des Pythagoras
- ◇ ursprünglich 2D



Inspirationen
aus dem Internet

Gedanke:
Beibehalten der Struktur aus Zweidimensionalen,
erweitern um dritte Ebene (Tiefe)

Serielle Umsetzung - Klassen

```
▼ /** Class Cuboid.  
 * Defines a cuboid by its eight corners like this:  
 *   H -- G  
 *  /      /|  
 * E -- F |  
 * |      | |  
 * | D --| C  
 * A -- B  
 * Side-lengths: AB = width / BC = depth / AE = height  
 */  
▼ class Cuboid {  
public:  
    // Rule of five:  
    Cuboid(const Cuboid&) = default;           // Copy-Constructor  
    Cuboid(Cuboid&&) = default;                 // Move-Constructor  
    Cuboid& operator=(const Cuboid&) = default; // Copy-Zuweisung  
    Cuboid& operator=(Cuboid&&) = default;      // Move-Zuweisung  
    ~Cuboid() = default;                       // Destructor  
  
    // further Constructors  
    Cuboid();  
    Cuboid(Point&, Point&, Point&, Point&, Point&, Point&, Point&, Point&);
```

- Überlegungen zu Aufbau Koordinatensystem, Strukturierung der geometrischen Figuren etc.
- Klassen:
Point, Triangle, Cuboid, Vector3D, Matrix

```
* Coordinate system:  
*   z   y  
*   |  /  
*   |/  
* ----- x  
*  /|  
*/
```

Serielle Umsetzung - Rekursion

```
//Builds the tree recursive and terminates if the side length of the cuboid is smaller than 0.05.
void buildTree(Cuboid cuboid, Renderer &renderer, double oldAngle) {
    double sideLength = cuboid.getTriangles()[0].computeSideLengths()[0];
    if (sideLength < 0.05) { // exit condition
        return;
    }

    // Definition of render object
    RenderObject cube;
    std::vector<Point> points = cuboid.getPoints();
    for (int i = 0; i < 8; i++) {
        cube.verts.push_back(points[i].toVector3D());
    }

    cube.edges = cuboid.getEdges();
    renderer.addRenderObject(cube);

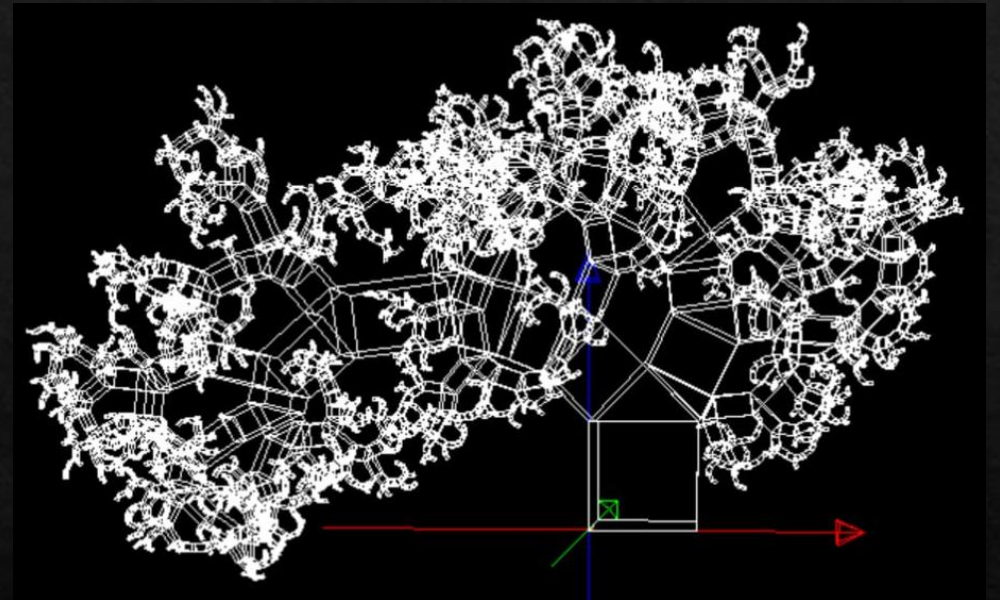
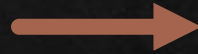
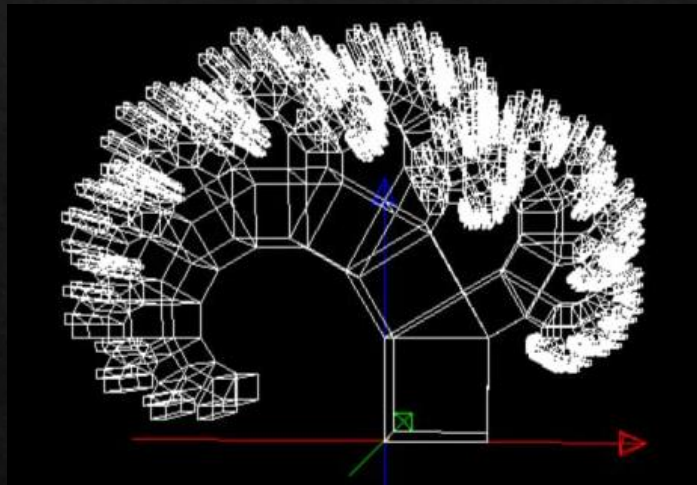
    // Computation of next Cuboids
    double randAngle = rand() % 45 + 5; // random factor
    double newAngle = oldAngle - randAngle; // new angle is old angle and random factor combined

    std::vector<double> newSideLength = getNewSideLength(points[3], points[2], randAngle);
    std::vector<double> vector_b = points[4].toStdVector();
    Cuboid newCuboid_b = Cuboid(getNextCuboid(newSideLength[1], newAngle, vector_b));

    std::vector<Point> cuboid_bPoints = newCuboid_b.getPoints();
    std::vector<double> vector_a = cuboid_bPoints[1].toStdVector();
    Cuboid newCuboid_a = Cuboid(getNextCuboid(newSideLength[0], newAngle + 90, vector_a));

    // Recursion
    buildTree(newCuboid_b, renderer, newAngle);
    buildTree(newCuboid_a, renderer, newAngle + 90);
}
```

Serielle Umsetzung – Randomisierung



// Quader statt Würfel:

```
double height = (rand() % 100 * (2*length)) / 100; // random height  
Cuboid newCuboid = Cuboid(height, length, depth);
```

// zufällige Winkel:

```
double randAngle = rand() % 45 + 5; // random faktor  
double newAngle = oldAngle - randAngle;
```


Parallelisierung - OpenCL

Schritt 1: aus Klassen werden structs

```
struct Point;

/* ... */
struct Vector3D {
    double x_;
    double y_;
    double z_;
};

// "Constructors"
Vector3D create_nullvector3D();
Vector3D create_vector3D_from_values(double, double, double);
Vector3D create_vector3D_from_points(Point, Point);

// functions
Vector3D normalize(Vector3D);
double getLengthOf3DVector(Vector3D);
Vector3D cross(const Vector3D&, const Vector3D&);
double scalarProduct(const Vector3D&, const Vector3D&);
Vector3D add_vector3D(const Vector3D&, const Vector3D&);
Vector3D subtract_vector3D(const Vector3D&, const Vector3D&);
Vector3D scalar_multiplication(const Vector3D&, float s);
```

```
struct Cuboid {
    Point points[8];
    int xedges[12];
    int yedges[12];
    double angle;
};

double* getSidelength(Cuboid);
Cuboid transform(Cuboid, double, char, double*);
Cuboid create_cuboid_from_sidelenght(double height,
    double width, double depth);

struct Vector3D;

struct Point {
    double x;
    double y;
    double z;
};

Point addVectorToPoint(Point point, double*);
Point transform3D(Point point, double, char, double*);
Vector3D toVector3D(Point p);
double* toArray(Point p);
```

Schritt 2: aus buildTree() wird generetateWithOpenCL()

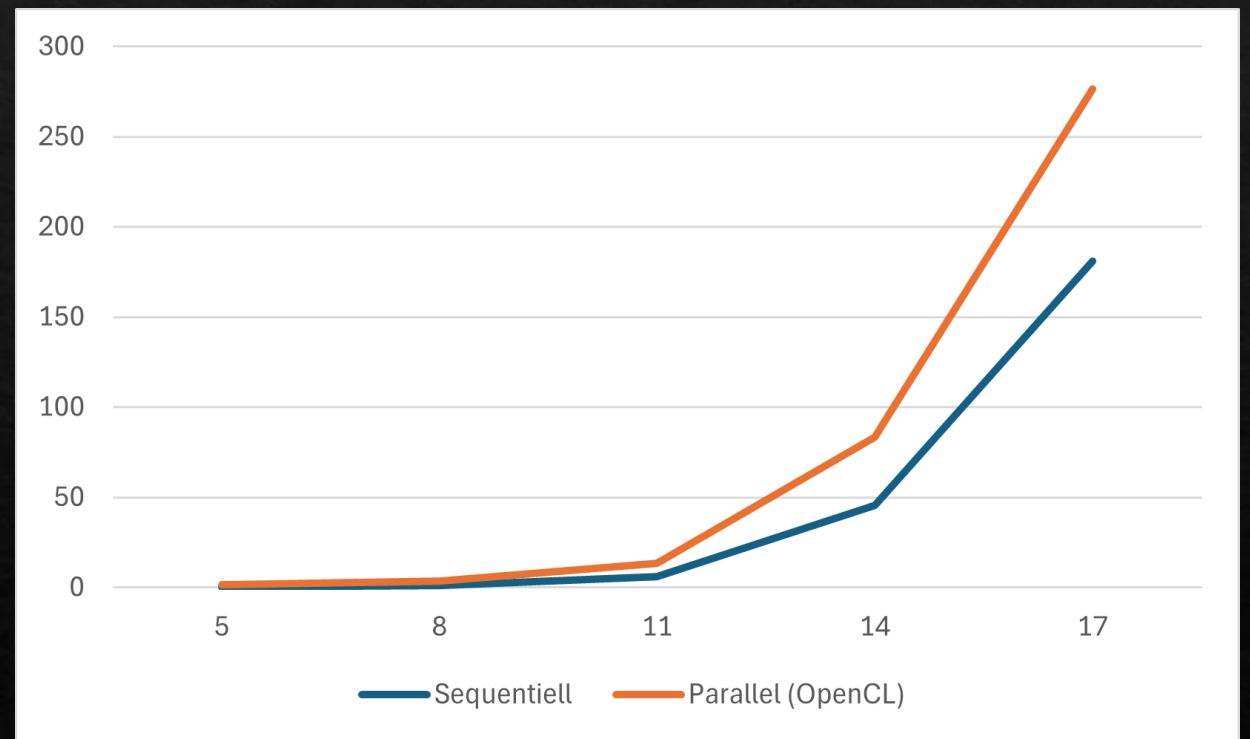


[Have a look!](#)

Parallelisierung – OpenCL: Speed-Up

Ebenen	Zeit sequentiell	Zeit parallel (OpenCL)
5	0,11775	1,344
8	0,83975	2,63975
11	5,987	7,2545
14	45,53075	37,78425
17	181,06925	95,4645

Messung:
je 4 Durchläufe pro Tiefe, Wert
anschließend gemittelt



Parallelisierung - OpenMP

- ◇ deutlich kürzer und einfacher als OpenCL
- ◇ aus `buildTree()` wird `generateWithOpenMP()`



[Have a look!](#)

Parallelisierung - OpenMP + OpenCL

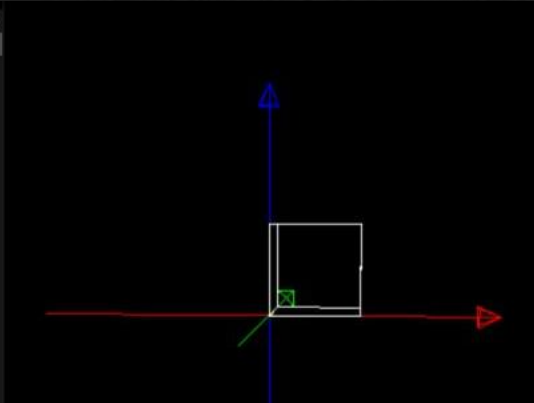
- ◇ Erster Teil des Baums wird mit OpenMP berechnet, Ebenen mit mehr Würfeln mit OpenCL
- => Ziel: maximaler Speedup durch Reduzieren des Kopierens von Daten auf die CPU

- ◆ Optimalen Punkt des Wechsels zwischen OpenMP und OpenCL bestimmen
- ◆ ggfs. Lookup-Table für Sinusberechnungen auf der GPU anlegen
- ◆ Evtl. MPI umsetzen
- ◆ Doku

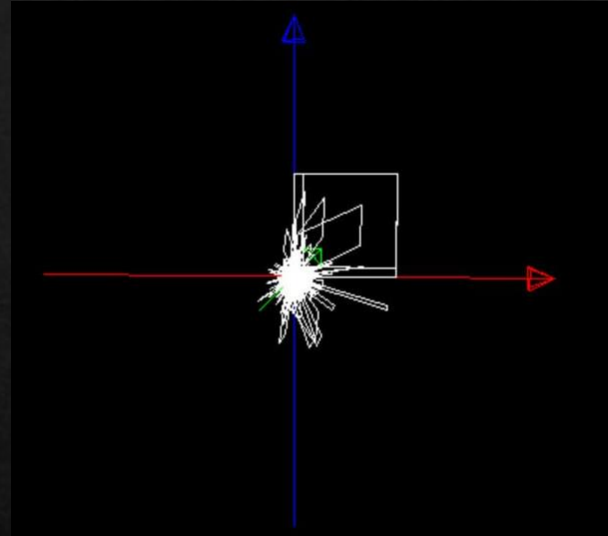
Weiterer Plan

Nobody is perfect - Outtakes

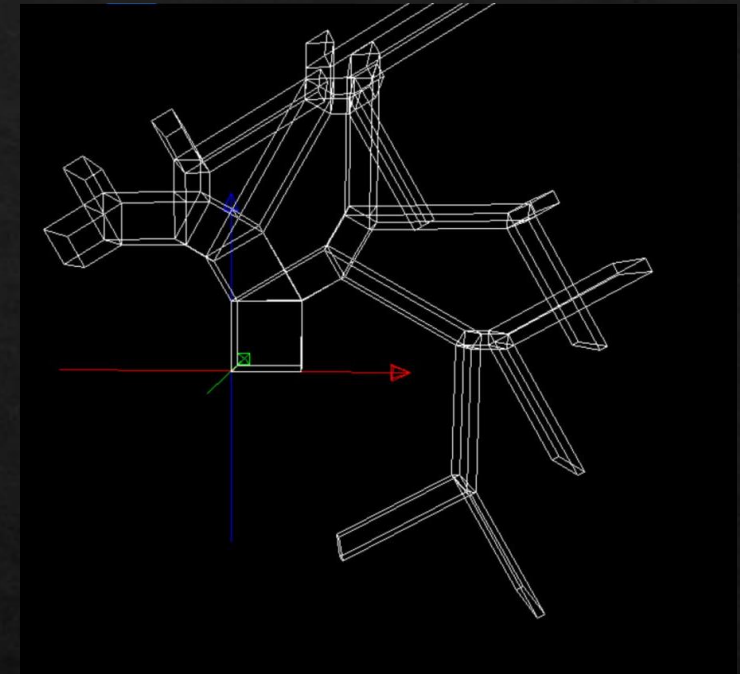
```
Detected OpenCL platforms: 1
AMD Accelerated Parallel Processing
Detected devices: 1
gfx902
buffer: 0000000000000000
error: 0
Kontext Info: 0000015D771E6290
Queue: 0000015D771EF7D0
Build status:
Result: 0 0 0
Result: 1.5 0 0
Result: 1.5 0 0
Result: 0 0 0
Result: 0 0 0
Result: 1.5 0 0
Result: 1.5 0 0
Result: 0 0 0
Execution time sequenziell: 0 seconds
```



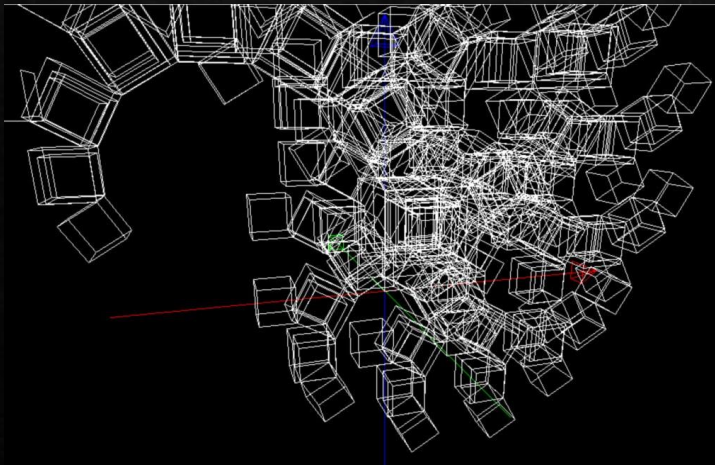
Der Baum ist weg
– aber immerhin steht nicht überall 0!



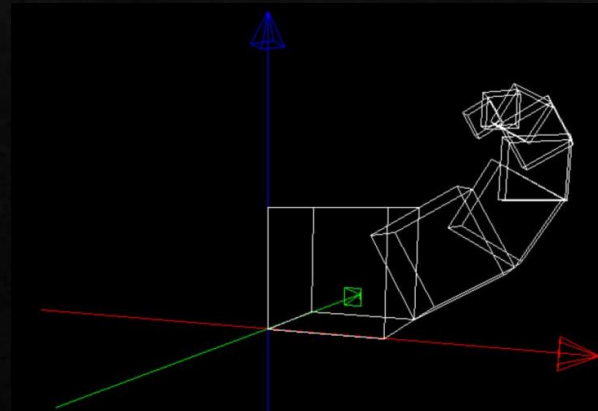
„Ups, ich habe den Baum
komprimiert“



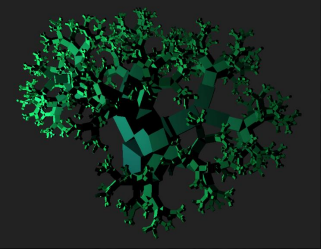
Moderne Kunst?



Naja,
so ähnlich...



Quellen



<https://codepen.io/jllodra/pen/LYqYqW> [Di, 20.01.2026]



<https://sketchfab.com/3d-models/3d-pythagoras-tree-1e58cfa7af294b2c9f314a2f04dcf823> [Di, 20.01.2026]